Document Ingestion

Document ingestion is the process of taking a user's uploaded project, extracting useful information from its files, preparing that information for RAG, and storing it in a form that DevDocs can later search and retrieve.

The goal is to transform a raw software project into a structured, searchable knowledge base.

---

1. What Is Document Ingestion?

Suppose a user uploads a project like this:

```
MyProject/
├── backend/
│   ├── main.py
│   ├── auth.py
│   └── database.py
├── frontend/
│   ├── App.jsx
│   └── components/
├── docs/
│   ├── architecture.md
│   └── api.md
├── README.md
└── requirements.txt
```

DevDocs cannot directly send this entire folder to an LLM and expect the LLM to understand it.

Instead, the project goes through an ingestion pipeline:

```
Project Folder
      ↓
File Discovery
      ↓
File Filtering
      ↓
Change Detection
      ↓
File Parsing
      ↓
Content Normalization
      ↓
Chunking
      ↓
Embedding
      ↓
```

```
Vector Database
      ↓
Ready for Retrieval
```

After ingestion, the project becomes a searchable knowledge base that can be used by the RAG pipeline.

---

2. Why Does DevDocs Need Document Ingestion?

DevDocs is not intended to be a simple chatbot.

Its purpose is to understand a software project.

For example, after ingestion, a user should eventually be able to ask:

- Where is authentication implemented?
- Which API is called when the user logs in?
- Where is the database connection created?
- Explain how the login flow works.
- Which files depend on `auth.py`?
- Where is payment processing implemented?
- What database does this project use?
- What does `main.py` do?
- How does a request travel from the frontend to the backend?
- What should I modify if I want to change the login behavior?

To answer these questions, DevDocs needs access to the project's information.

Document ingestion is the process that makes this information searchable.

---

3. What Types of Files Can Be Uploaded?

DevDocs should support both **documentation** and **source-code files**.

Recommended File Types

Category	Extensions	Priority
Markdown	`.md`	High
PDF	`.pdf`	High
Source Code	`.py`, `.java`, `.js`, `.jsx`, `.ts`, `.tsx`	High
Text	`.txt`	High
Word Documents	`.docx`	Medium

Category	Extensions	Priority
HTML	`.html`	Medium
JSON	`.json`	Medium
YAML	`.yaml`, `.yml`	Medium
CSV	`.csv`	Medium

The most important formats for the first version are:

1. Markdown
2. PDF
3. Source code
4. TXT

Additional formats can be added later.

---

4. Uploading an Entire Project

The user should not have to upload every file individually.

Instead, DevDocs should allow the user to upload an **entire project folder**.

For example:

```
MyProject/
├── backend/
│   ├── main.py
│   ├── auth.py
│   └── database.py
├── frontend/
│   ├── App.jsx
│   └── components/
├── docs/
│   ├── architecture.md
│   └── api.md
├── README.md
└── requirements.txt
```

The frontend sends the project files to the backend while preserving their relative paths.

For example:

```
backend/main.py
backend/auth.py
frontend/App.jsx
```

```
docs/architecture.md
README.md
```

The relative path is important because DevDocs needs to know where a file belongs inside the project.

---

5. Why Upload the Entire Folder?

A software project is not simply a collection of unrelated files.

Files have relationships with each other.

For example:

```
frontend/Login.jsx
        ↓
      calls
        ↓
backend/auth.py
        ↓
      uses
        ↓
database.py
```

If DevDocs only receives `Login.jsx`, it may not have enough context to understand the complete implementation.

By ingesting the entire project, DevDocs can eventually understand relationships between:

- frontend files
- backend files
- configuration files
- documentation
- APIs
- database code
- business logic
- utility functions

This makes the application more useful than a generic document chatbot.

---

6. Greenfield Projects

DevDocs should also work when a project does not have a codebase yet.

A greenfield project might initially contain:

```
MyProject/
├── README.md
├── requirements.md
├── architecture.md
└── api-spec.yaml
```

There may be no source code yet.

DevDocs can still ingest these documents and answer questions such as:

- What is this project supposed to do?
- What APIs are required?
- What database should be used?
- What are the system requirements?
- How should authentication work?
- What components need to be implemented?

Later, when development begins, the user can upload the updated project again.

Therefore, DevDocs can evolve with the project:

```
Greenfield Project
        ↓
Requirements
        ↓
Architecture
        ↓
Development
        ↓
Source Code
        ↓
Maintenance
```

---

7. Recommended File Size

For the first version, reasonable limits could be:

```
Maximum individual file size: 10–20 MB
Maximum project upload: approximately 50 MB
```

These are application-level limits rather than technical requirements.

The important consideration is not only file size.

For example:

```
10 MB README.md
```

could contain useful information.

Whereas:

```
10 MB generated build files
```

may provide almost no value.

Therefore, file filtering is just as important as file-size limits.

---

8. Files and Directories to Ignore

Real software projects contain many files that should not be ingested.

Examples:

```
.git/
node_modules/
venv/
.venv/
__pycache__/
dist/
build/
target/
coverage/
.env
```

These files may be:

- dependencies
- generated files
- build artifacts
- temporary files
- cache files
- sensitive configuration
- irrelevant to understanding the application

DevDocs should maintain an ignore list similar to `.gitignore`.

Example:

```
.git/
node_modules/
venv/
.venv/
__pycache__/
dist/
build/
.env
```

Ignoring these directories provides several benefits:

- Less processing
- Less chunking
- Fewer embeddings
- Smaller vector database
- Less retrieval noise
- Faster ingestion

---

9. Complete Ingestion Pipeline

The complete pipeline is:

```
User Uploads Project
        ↓
File Discovery
        ↓
File Filtering
        ↓
Change Detection
        ↓
File-Type Specific Parsing
        ↓
Content Normalization
        ↓
Chunking
        ↓
Embedding
        ↓
Vector Database
        ↓
Metadata Database
        ↓
Ready for RAG
```

Each stage has a specific responsibility.

---

10. Step 1 — File Discovery

The backend first identifies all files inside the uploaded project.

For example:

```
MyProject/
├── backend/
│   ├── main.py
│   └── auth.py
├── frontend/
│   └── App.jsx
└── README.md
```

The backend creates a list such as:

```
backend/main.py
backend/auth.py
frontend/App.jsx
README.md
```

These paths become the initial representation of the uploaded project.

---

11. Step 2 — File Filtering

Before processing the files, DevDocs determines which files should be ignored.

For example:

```
node_modules/react/index.js
```

should not normally be processed.

The backend filters it out.

Only relevant files continue through the ingestion pipeline.

---

12. Step 3 — Change Detection

Change detection is one of the most important parts of the ingestion architecture.

The user may upload the **entire project again** after making changes.

Suppose the first upload contains:

```
backend/
├── main.py
├── auth.py
└── database.py
```

Later, the user changes only:

```
auth.py
```

We do not want to process everything again.

Instead:

```
main.py       → unchanged → SKIP
auth.py       → changed   → REPROCESS
database.py   → unchanged → SKIP
```

This is called **incremental ingestion**.

---

13. Why Incremental Ingestion Is Important

A naive implementation could do this every time:

```
Upload project
      ↓
Delete all existing data
      ↓
Parse every file
      ↓
Chunk every file
      ↓
Embed every file
      ↓
Store everything again
```

This works, but it is inefficient.

It causes:

- unnecessary processing
- unnecessary embedding requests
- slower ingestion

- unnecessary vector database operations
- potentially higher API costs

Instead, DevDocs should process only what has changed.

```
Upload project
      ↓
Compare with previous version
      ↓
Process only changed/new files
```

---

14. File Hashing

DevDocs can use file hashing to determine whether the contents of a file have changed.

A hash is essentially a fingerprint of the file's contents.

For example:

```
auth.py
   ↓
SHA-256
   ↓
8a6f...29bd
```

DevDocs stores this hash with the file metadata.

For example:

```
path: backend/auth.py
hash: 8a6f...29bd
```

When the project is uploaded again, DevDocs calculates the new hash.

---

15. Unchanged File

Suppose the previous version contains:

```
auth.py
hash = ABC123
```

The new upload contains:

```
auth.py
hash = ABC123
```

The hashes are identical.

Therefore:

```
Old Hash == New Hash
        ↓
File contents are unchanged
        ↓
Skip ingestion
```

There is no reason to parse, chunk, or embed the file again.

---

16. Changed File

Suppose the previous version contains:

```
auth.py
hash = ABC123
```

The new version contains:

```
auth.py
hash = XYZ789
```

The hashes are different.

Therefore:

```
Old Hash != New Hash
        ↓
File has changed
        ↓
Delete old chunks
        ↓
Parse new file
        ↓
Chunk new content
        ↓
Generate new embeddings
```

```
        ↓
Store new chunks
```

---

17. Why Does Hashing Detect Changes?

A cryptographic hash such as SHA-256 generates a fingerprint based on the contents of the file.

Even a small content modification can result in a different hash.

For example:

```
return user
```

might produce:

```
ABC123...
```

while:

```
return current_user
```

might produce:

```
F91D82...
```

DevDocs does not need to understand what changed.

It only needs to compare the fingerprints:

```
Old Hash == New Hash
        ↓
No content change

Old Hash != New Hash
        ↓
Content changed
```

---

18. Does Hashing Cost Money?

No.

SHA-256 hashing can be performed locally by the computer.

It does not require:

- an API call
- an external service
- an LLM
- an embedding provider
- a cloud service

Therefore:

```
Hashing = ₹0
```

Hashing does consume CPU and requires reading the file, but it is generally inexpensive compared with embedding and LLM operations.

---

19. What Happens With Huge Files?

Hashing a huge file does require reading its contents.

However, the entire file does not need to be loaded into memory.

The file can be processed in small chunks.

For example:

```
5 GB File
   ↓
Read 1 MB
   ↓
Update SHA-256
   ↓
Read next 1 MB
   ↓
Update SHA-256
   ↓
...
   ↓
Final Hash
```

This keeps memory usage low.

Conceptually:

```
hasher = hashlib.sha256()

with open(file_path, "rb") as file:
    while chunk := file.read(1024 * 1024):
        hasher.update(chunk)

file_hash = hasher.hexdigest()
```

The important idea is:

```
Do not load the entire file into memory just to calculate its hash.
```

Hashing is also much cheaper than unnecessarily re-embedding the same content.

---

20. File Metadata

DevDocs should maintain metadata for every ingested file.

A file record can contain:

```
project_id
file_path
file_name
file_type
file_size
file_hash
last_modified
ingestion_status
```

Example:

```
{
  "project_id": 1,
  "file_path": "backend/auth.py",
  "file_name": "auth.py",
  "file_type": "python",
  "file_size": 18240,
  "file_hash": "8a6f...29bd",
```

```
  "ingestion_status": "completed"
}
```

This information is useful during future ingestion operations.

---

21. File Size and Modification Time

Hashing is reliable for determining content changes, but the application can use other metadata to optimize the process.

For example:

```
file_path
file_size
last_modified
file_hash
```

Suppose the previous version contains:

```
auth.py
size = 18 KB
modified = 10:30
hash = ABC123
```

The new upload contains:

```
auth.py
size = 18 KB
modified = 10:30
```

The application can potentially avoid calculating the hash because the metadata is identical.

If metadata indicates a possible change:

```
auth.py
size = 19 KB
modified = 14:20
```

then the application can calculate the SHA-256 hash to confirm whether the content changed.

The hash remains the reliable content-level check.

---

22. Important Limitation of Hashing

Hashing helps with **processing efficiency**, but it does not automatically reduce the amount of data uploaded.

If the user uploads the entire folder again:

```
Entire Project
      ↓
Uploaded to Backend
      ↓
Hash Comparison
```

The unchanged files have still been transferred.

Hashing prevents unnecessary:

- parsing
- chunking
- embedding
- vector database updates

It does not automatically prevent network transfer.

A future GitHub-based or synchronization-based implementation could avoid uploading unchanged files altogether.

---

23. Detecting New Files

Suppose the previous project contains:

```
main.py
auth.py
```

The new project contains:

```
main.py
auth.py
payment.py
```

`payment.py` did not exist previously.

Therefore:

```
payment.py
    ↓
NEW FILE
    ↓
Process
```

---

24. Detecting Modified Files

Suppose:

```
auth.py
```

exists in both versions.

Previous:

```
auth.py → hash ABC
```

New:

```
auth.py → hash XYZ
```

Therefore:

```
auth.py
    ↓
MODIFIED
    ↓
Reprocess
```

---

25. Detecting Deleted Files

Suppose the previous project contains:

```
main.py
auth.py
database.py
```

The new project contains:

```
main.py
auth.py
```

`database.py` is missing.

Therefore:

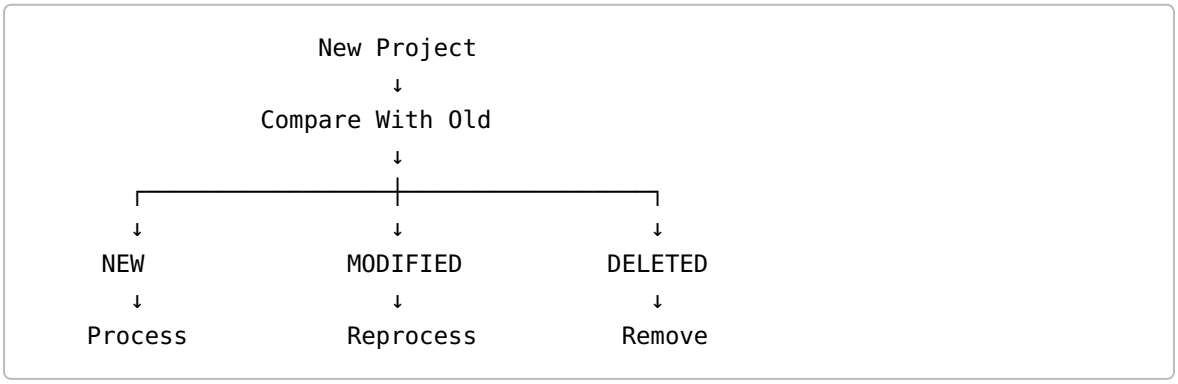
```
database.py
    ↓
DELETED
    ↓
Remove its chunks from the vector database
    ↓
Remove its metadata
```

This is important because otherwise DevDocs could retrieve information from a file that no longer exists in the current version of the project.

---

26. Complete Change Detection

The result of comparing two project versions can be represented as:



Files that are neither new, modified, nor deleted are unchanged and can be skipped.

```
UNCHANGED
    ↓
SKIP
```

---

27. File Rename or Move

File paths should also be stored.

For example:

```
backend/auth.py
```

is different from:

```
backend/security/auth.py
```

even if both files contain identical content.

A simple V1 implementation can treat this as:

```
Old path
    ↓
Deleted

New path
    ↓
New file
```

This means the file may be processed again.

A future optimization could compare hashes and detect:

```
Same Hash
Different Path
        ↓
Possible Rename/Move
```

This is not necessary for V1.

---

28. Step 4 — File-Type Specific Parsing

Once DevDocs knows which files need processing, it extracts their content.

Different file types require different parsing strategies.

```
PDF   → PDF Parser
DOCX  → DOCX Parser
HTML  → HTML Parser
MD    → Markdown Parser
Code  → Code/Text Parser
JSON  → JSON Parser
```

```
YAML  → YAML Parser
TXT   → Text Parser
```

The goal is to convert different file formats into a common internal representation.

---

29. PDF Parsing

For PDFs, DevDocs should extract:

- text
- page information
- useful document metadata

Conceptually:

```
PDF
 ↓
PDF Parser
 ↓
Text + Page Metadata
```

Example metadata:

```
source = architecture.pdf
page = 12
```

Page information is valuable because the final RAG answer can later identify where the information came from.

---

30. Markdown Parsing

Markdown contains useful structure.

For example:

```
# Authentication

## Login

The login endpoint validates the user's credentials.

## JWT
```

```
The application uses JWT for authentication.
```

Instead of treating everything as plain text, DevDocs can preserve headings.

For example:

```
document = authentication.md
section = Authentication
subsection = Login
```

This metadata can improve retrieval and citations.

---

31. Source Code Parsing

Source code should preserve information such as:

- file path
- programming language
- functions
- classes
- methods
- imports
- comments
- logical structure

For example:

```
backend/auth.py
        ↓
Python Parser
        ↓
validate_token()
create_token()
authenticate_user()
```

Eventually, DevDocs can use syntax-aware or AST-based parsing.

For V1, a simpler language-aware text-splitting approach can be used.

---

32. DOCX Parsing

DOCX documents can be converted into text while preserving useful structure.

Conceptually:

```
DOCX
 ↓
DOCX Parser
 ↓
Extract Text
 ↓
Preserve Headings / Structure
```

The resulting content can then move into the common ingestion pipeline.

---

33. JSON and YAML Parsing

JSON and YAML are structured data formats.

For example:

```
{
  "database": {
    "type": "postgresql",
    "host": "localhost"
  }
}
```

The parser should preserve the structure instead of blindly treating it as arbitrary text.

This is especially useful for:

- configuration files
- API specifications
- application settings
- deployment configuration

---

34. Common Internal Representation

Although different file types require different parsers, their output should eventually be converted into a common internal representation.

Conceptually:

```
Document
├── content
```

```
├── source
├── file_path
├── file_type
└── metadata
```

Example:

```
{
  "content": "The login endpoint validates the JWT...",
  "source": "backend/auth.py",
  "file_path": "backend/auth.py",
  "file_type": "python",
  "metadata": {
    "project_id": 1,
    "function": "validate_token"
  }
}
```

Once files reach this common representation, the downstream RAG pipeline can process them consistently.

---

35. Step 5 — Chunking

A complete file should generally not be embedded as one enormous piece.

Instead, its content is divided into smaller chunks.

For example:

```
auth.py
   ↓
Chunk 1
Chunk 2
Chunk 3
Chunk 4
```

The chunks should ideally represent meaningful pieces of information.

---

36. Chunking Different File Types

Different types of content benefit from different chunking strategies.

Markdown

Good boundaries include:

```
Heading
Section
Subsection
```

Source Code

Good boundaries include:

```
Function
Class
Method
Logical code block
```

PDF

Good boundaries include:

```
Section
Paragraph
Page content
```

Plain Text

A recursive or token-based text splitter can be used.

The goal is to avoid splitting information at meaningless boundaries.

---

37. Step 6 — Embeddings

Each chunk is converted into an embedding.

Conceptually:

```
Text Chunk
    ↓
Embedding Model
    ↓
Vector
```

For example:

```
"JWT token is validated using..."
        ↓
[0.021, -0.184, 0.732, ...]
```

The embedding represents the semantic meaning of the chunk.

This allows DevDocs to later find chunks that are semantically related to a user's question.

---

38. Step 7 — Vector Database

The generated embeddings are stored in a vector database.

A vector record should contain more than just the embedding.

Conceptually:

```
Vector Record
├── embedding
├── chunk_text
└── metadata
```

Example:

```
{
  "embedding": [0.021, -0.184, 0.732],
  "text": "JWT token is validated using...",
  "metadata": {
    "project_id": 1,
    "file_path": "backend/auth.py",
    "file_type": "python",
    "function": "validate_token",
    "chunk_id": "auth_003"
  }
}
```

The metadata is important because DevDocs needs to know where the retrieved information came from.

---

39. Why Metadata Is Important

Suppose a user asks:

> Where is JWT validation implemented?

The vector search might return:

```
Chunk:
"JWT token is validated using validate_token()..."

Metadata:
file_path = backend/auth.py
function = validate_token
```

DevDocs can then answer:

```
JWT validation is implemented in
backend/auth.py inside validate_token().
```

Without metadata, the system may know the content but not where it came from.

---

40. Where Should Original Files Be Stored?

The vector database should not be treated as the original project storage.

There are three logical types of data:

```
Original Files
      ↓
File / Object Storage

File Metadata
      ↓
Relational Database

Embeddings + Chunks
      ↓
Vector Database
```

The exact implementation can be simplified for V1, but separating these responsibilities makes the architecture easier to maintain.

---

41. Metadata Database

A relational database can store information about:

```
Project
File
File Hash
File Path
File Type
File Size
Ingestion Status
```

Example:

```
Project
   ↓
Files
   ├── main.py
   ├── auth.py
   ├── database.py
   └── README.md
```

This makes incremental ingestion easier.

---

42. Vector Database

The vector database stores:

```
Chunk
Embedding
Chunk Metadata
```

Conceptually:

```
Project
   ↓
File
   ↓
Chunks
   ↓
Embeddings
```

The metadata database answers questions such as:

> What files exist in this project?

The vector database answers questions such as:

Which pieces of content are semantically related to this user's question?

---

43. Re-ingestion Example

Consider the first project upload:

```
project/
├── backend/
│   ├── main.py
│   ├── auth.py
│   └── database.py
├── frontend/
│   └── App.jsx
└── README.md
```

DevDocs processes all relevant files.

Later, the user:

- modifies `auth.py`
- adds `payment.py`
- deletes `database.py`
- leaves everything else unchanged

The new project is:

```
project/
├── backend/
│   ├── main.py
│   ├── auth.py
│   └── payment.py
├── frontend/
│   └── App.jsx
└── README.md
```

DevDocs determines:

```
main.py       → unchanged → SKIP
auth.py       → modified  → REPROCESS
database.py   → deleted   → REMOVE
payment.py    → new       → PROCESS
App.jsx       → unchanged → SKIP
README.md     → unchanged → SKIP
```

Only the necessary work is performed.

44. Reprocessing a Modified File

When a file changes, the old chunks should not remain in the vector database.

For example:

```
Old auth.py
    ↓
Old chunks
    ↓
Old embeddings
```

After the file changes:

```
New auth.py
    ↓
New chunks
    ↓
New embeddings
```

Therefore:

```
Delete old chunks
        ↓
Parse new file
        ↓
Create new chunks
        ↓
Create new embeddings
        ↓
Store new chunks
```

Otherwise, DevDocs could retrieve outdated information.

45. Handling Deleted Files

If a file disappears from the new project upload, its previous information must also be removed.

Example:

```
Old:
database.py
```

```
New:
database.py does not exist
```

DevDocs should:

```
Delete file metadata
        ↓
Delete associated chunks
        ↓
Delete associated embeddings
```

This ensures that retrieval represents the **current project state**.

---

46. Ingestion Status

Ingestion can take time, especially for large projects.

Therefore, DevDocs should track ingestion status.

Possible statuses:

```
pending
processing
completed
failed
```

Example:

```
Project Upload
      ↓
pending
      ↓
processing
      ↓
completed
```

If something goes wrong:

```
processing
      ↓
failed
```

This makes it easier for the UI to show the user what is happening.

47. Error Handling

Different files can fail for different reasons.

Examples:

- corrupted PDF
- unsupported encoding
- invalid DOCX
- unreadable file
- extremely large file
- unsupported extension
- embedding API failure

The ingestion pipeline should not necessarily fail for the entire project because one file failed.

For example:

```
main.py       → completed
auth.py       → completed
README.md     → completed
broken.pdf    → failed
```

The UI can then report:

```
Ingestion completed with 1 failed file.
```

This is more robust than stopping the entire ingestion process.

---

48. V1 Ingestion Strategy

The first version of DevDocs should remain relatively simple.

V1 should support:

- Entire project folder upload
- File-level change detection
- SHA-256 hashing
- File metadata storage
- Basic file-type parsing
- Content normalization
- Text/code chunking
- Embeddings
- Vector database

- Ignoring unnecessary directories
- Incremental ingestion
- New file detection
- Modified file detection
- Deleted file detection
- Basic ingestion status

---

49. What V1 Should Not Try to Solve

The first version does not need to implement everything.

Avoid making V1 unnecessarily complex with:

- Git integration
- GitHub synchronization
- AST-based analysis for every language
- chunk-level diffing
- dependency graph analysis
- branch-aware ingestion
- automatic code relationship detection
- real-time ingestion
- advanced background job orchestration

These can be added later.

The objective of V1 is:

> Upload a project, efficiently process its useful content, and make it searchable for RAG.

---

50. Future Improvements

Once the basic ingestion pipeline works, DevDocs can become significantly more advanced.

Potential improvements include:

GitHub Integration

Instead of uploading a folder manually:

```
GitHub Repository
       ↓
DevDocs
       ↓
Ingestion
```

DevDocs could periodically synchronize with the repository.

Automatic Re-ingestion

When the project changes:

```
Project Changed
      ↓
Detect Changes
      ↓
Re-ingest Automatically
```

Chunk-Level Change Detection

V1 can detect changes at the file level.

A future version could detect changes inside a file.

For example:

```
auth.py
├── login()
├── logout()
├── validate_token()
└── refresh_token()
```

If only `login()` changes, a future system could avoid re-embedding all other functions.

AST-Based Code Parsing

Instead of treating code purely as text, DevDocs could understand the actual structure of the code.

For example:

```
Python File
   ↓
AST
   ↓
Classes
Functions
Methods
```

```
Imports
Relationships
```

This would allow more intelligent code-aware retrieval.

Dependency Graph

DevDocs could eventually understand:

```
Login.jsx
    ↓
auth API
    ↓
auth.py
    ↓
database.py
```

This would allow questions such as:

> What files are involved in the login flow?

---

51. Cost Considerations

Hashing itself is free:

```
Hashing → ₹0
```

Parsing and chunking can also be performed locally:

```
Parsing  → ₹0
Chunking → ₹0
```

Potential costs come from:

```
Embedding API
LLM API
Cloud Storage
Cloud Vector Database
```

Since the DevDocs project has a target of **₹0 cost**, the implementation should prefer free hosted services where feasible and avoid unnecessary API calls.

Incremental ingestion is useful even when using a free embedding provider because it reduces unnecessary processing.

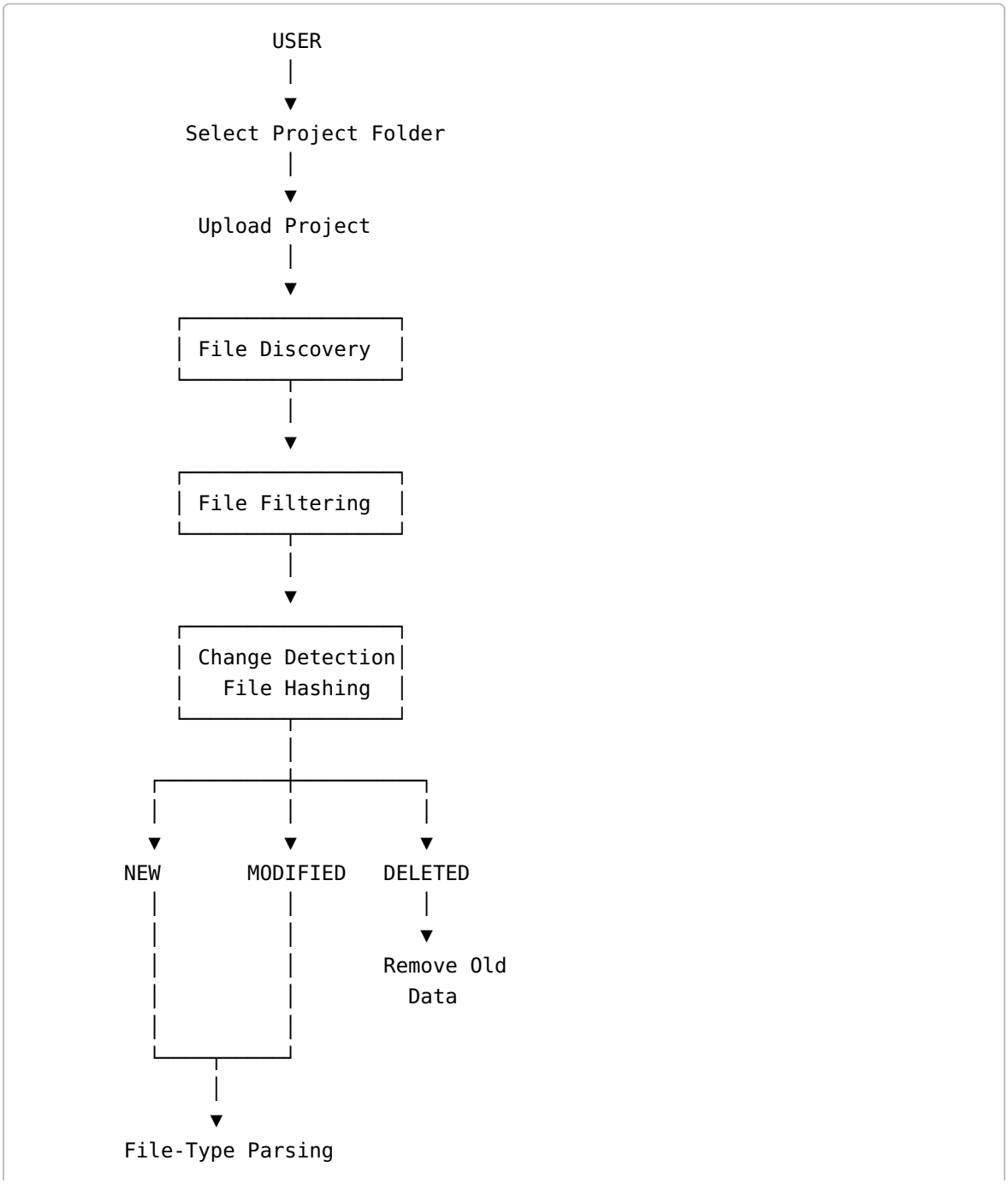
The core principle is:

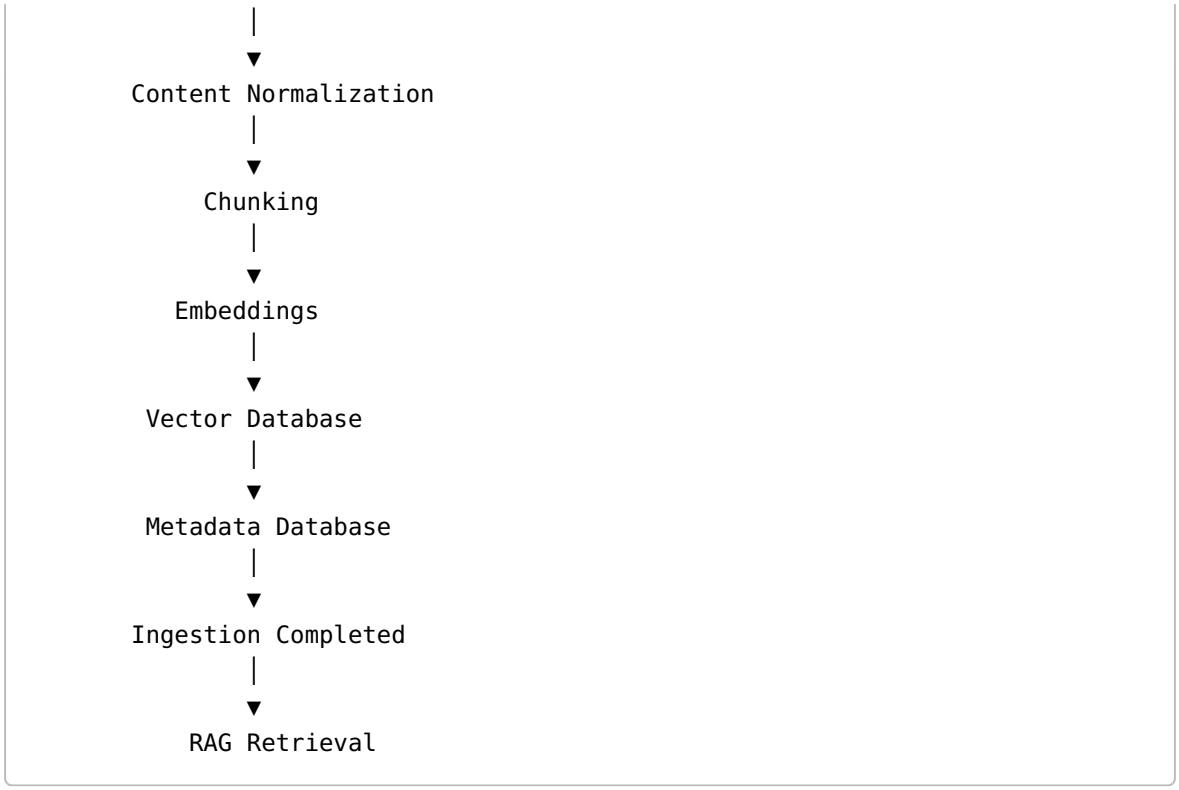
> Do not perform expensive work when it is not necessary.

---

52. Complete V1 Flow

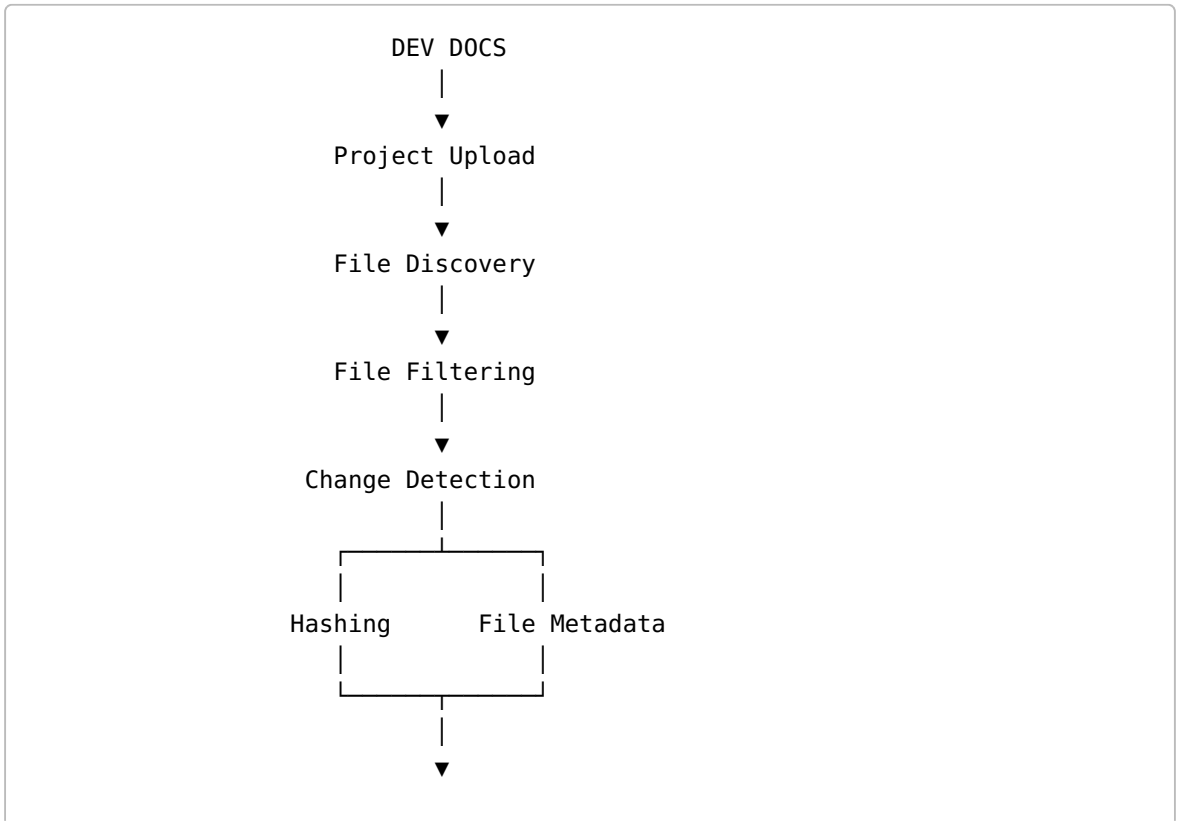
The complete V1 flow is:

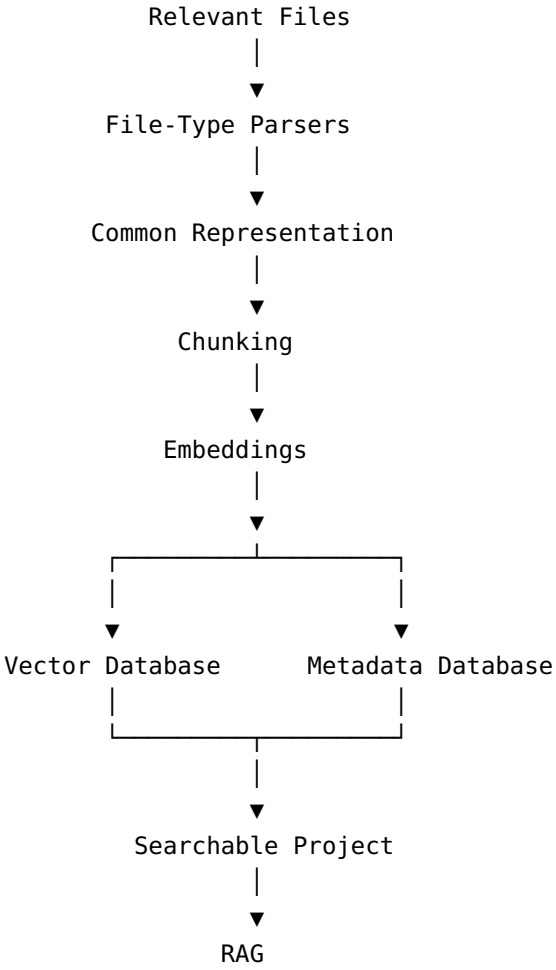




53. Final Architecture

The overall document ingestion architecture can be summarized as:





---

54. Key Design Principle

The most important idea behind DevDocs ingestion is:

> **Uploading the entire project does not mean processing the entire project.**

The user should have a simple experience:

```
Upload latest project
```

DevDocs should handle the complexity internally:

```
Compare old and new files
        ↓
Detect new files
        ↓
Detect modified files
        ↓
```

```
Detect deleted files
        ↓
Skip unchanged files
        ↓
Process only what is necessary
```

This gives the user a simple workflow while keeping the backend efficient.

---

55. What Document Ingestion Ultimately Achieves

Before ingestion:

```
Raw Project
```

After ingestion:

```
Structured Knowledge Base
```

Conceptually:

```
Project
│
├── Files
│   ├── main.py
│   ├── auth.py
│   ├── database.py
│   └── README.md
│
├── Metadata
│   ├── file paths
│   ├── file types
│   ├── file hashes
│   └── ingestion status
│
├── Chunks
│   ├── Chunk 1
│   ├── Chunk 2
│   ├── Chunk 3
│   └── ...
│
└── Embeddings
    ├── Vector 1
    ├── Vector 2
```

```
    ├── Vector 3
    └── ...
```

This knowledge base becomes the foundation for the next major part of DevDocs:

Retrieval and RAG.